NOTES — WEEK 4 & 5

Machine Learning & Linear Regression

Covers what ML actually is, the four types, full pipeline from data to deployment, linear regression from scratch, how gradient descent works, how to evaluate a model, and regularization.

01 What is Machine Learning?

The core idea is simple. In normal programming, you write rules — if this, do that. In machine learning, you don't write the rules. You give the system a large number of examples with correct answers, and it figures out the rules on its own. The more data, the better it gets.

The classic framing: instead of programming a spam filter by listing every spammy keyword, you show it 100,000 emails labeled "spam" or "not spam" and it learns what makes an email spam. You never write a single rule.

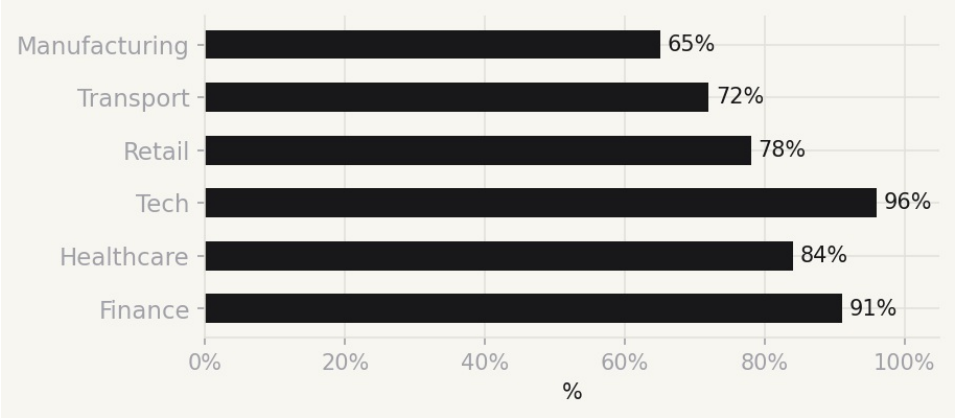
> *The name "machine learning" comes from Arthur Samuel (1959):* **"the ability to learn without being explicitly programmed."** *Still the best one-line definition.*

Where it actually matters

ML is not just a tech trend. It's in production everywhere — fraud detection in your bank app, the route Google Maps picks, Netflix recommendations, autocorrect, medical imaging diagnosis. Anything where the rules are too complex or too numerous to write by hand is a candidate for ML.

ML adoption by industry

% of large companies with ML in active production systems

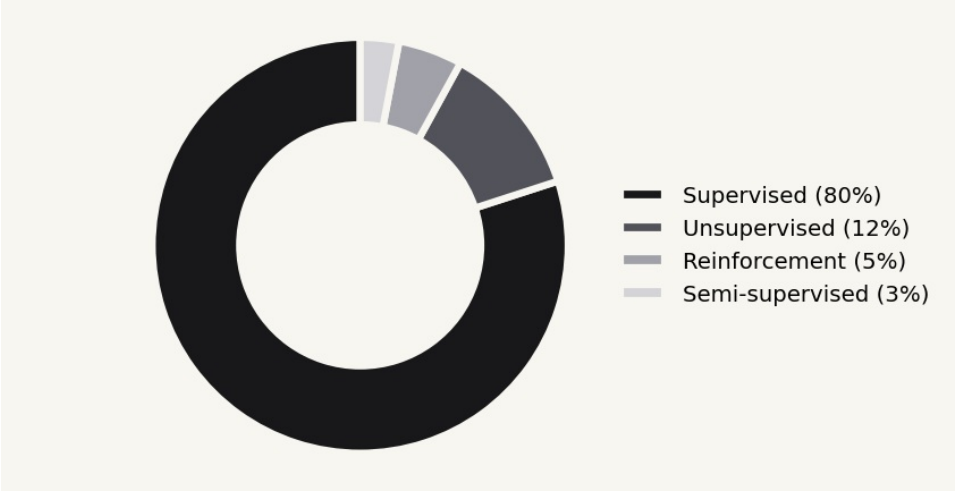


02 Types of Machine Learning

There are four types. In practice, ~80% of real ML work is supervised learning. The others exist but you see them less in standard industry work.

Share of real-world ML by type

Supervised dominates because labeled data exists in most business settings



Supervised Learning

You have labeled data — each input has a known correct output. The model learns to map input to output. Two sub-types:

- **Regression** — the output is a number. House price, temperature, sales volume.
- **Classification** — the output is a category. Spam/not spam, disease/no disease, digit recognition (0–9).

Unsupervised Learning

No labels. The algorithm finds structure in the data on its own — natural groupings, compressed representations, anomalies. Customer segmentation is a good example: you don't tell it what the groups are, it finds them.

Reinforcement Learning

An agent takes actions, gets rewards for good ones and penalties for bad ones, and learns through trial and error. Used in game AI, robotic control, self-driving. Less common in day-to-day data science work.

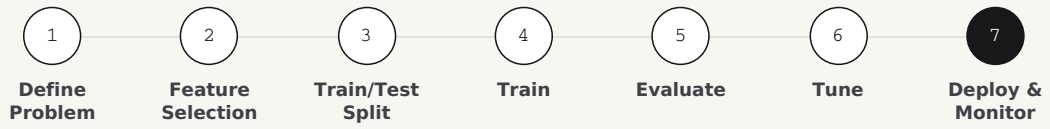
Semi-Supervised

A small amount of labeled data plus a large amount of unlabeled. Used when labeling is expensive — medical imaging requiring expert annotation, for example.

> *Binary classification = two classes (yes/no). Multi-class = three or more (cat, dog, bird, fish). The algorithms work the same — just extended for more outputs.*

03 The ML Workflow

Every ML project follows the same sequence. Each step depends on the one before it. Rushing through feature selection or skipping proper train/test splitting creates problems that are very hard to diagnose later.



Step 1 — Define the problem. Decide what you're predicting before touching any data. Number → regression. Category → classification. This determines which algorithms, which metrics, and which loss function you use.

Step 2 — Feature selection. Only keep columns with a real relationship to the output. For house prices: size, bedrooms, location — yes. Owner name, paint color — no. Useless features add noise. Correlated features cause multicollinearity.

Step 3 — Train/test split. Hold out a portion of data (usually 20%) that the model never sees during training. This is the only honest way to measure real performance. Testing on training data is cheating — the model just memorizes the answers.

> *Standard split: 80/20. With hyperparameter tuning, use 60/20/20 (train/validation/test) or k-fold cross-validation to avoid using test data during tuning.*

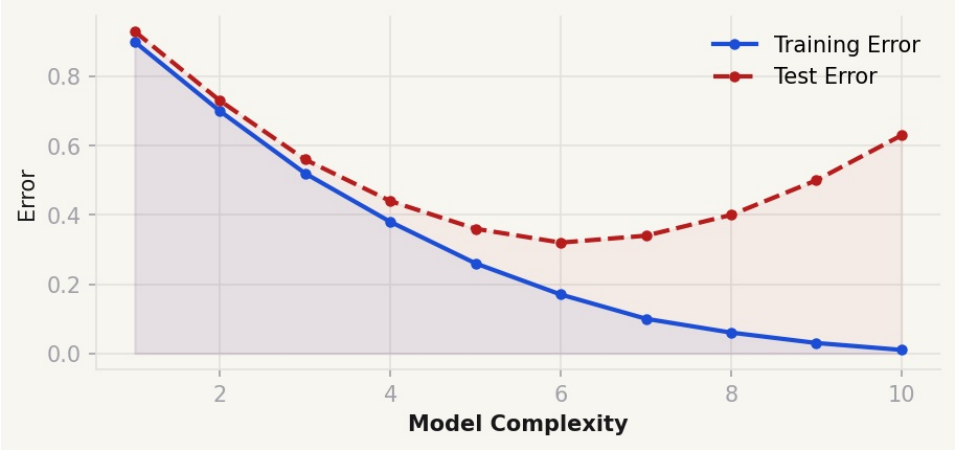
Steps 4–6 — Train, evaluate, tune. Feed training data to the model, measure performance on test data using your chosen metrics (MSE, R^2, etc.), then adjust hyperparameters and repeat. Tuning is done on the validation set — never the test set.

Step 7 — Deploy and monitor. Ship to production. Real-world data changes over time. If the distribution shifts (model drift), performance degrades and you need to retrain. This is why monitoring matters.

> **Overfitting vs Underfitting**
>
> Overfitting: model memorizes training data, fails on new data. Training error low, test error high. Too complex.
> Underfitting: model is too simple, performs poorly on both. Increase complexity or add features.
> The goal is low error on both. The gap between training and test error is the signal.

Training error vs test error as model complexity increases

The gap between the two lines is the overfitting signal — find the complexity where test error is minimum



04 Linear Regression

Supervised learning for predicting a continuous numeric value. The model assumes a linear relationship between the inputs and output, and finds the straight line (or hyperplane for multiple inputs) that minimizes prediction error across all training points.

SIMPLE LINEAR REGRESSION

$$y = mx + c$$

y — predicted output · **x** — input feature · **m** — slope (how much y changes per unit of x) · **c** — intercept (value of y when x = 0)

MULTIPLE FEATURES

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

b_0 — intercept · **b_1, b_2...** — how much each feature contributes · **x_1, x_2...** — input features

The model's job is to find the values of m and c (or b_0, b_1...) that produce the smallest average prediction error across all training examples. That error is measured with MSE.

A **residual** is the difference between actual and predicted for one data point: `residual = actual - predicted`. The line that minimizes the sum of all squared residuals is the best fit line.

House size vs price — best fit line through the data

Each point is one house. The line is the model. The vertical gap between any point and the line is the residual for that house.

Interpreting coefficients

In a model like `Sales = 2.9 + 0.046·TV + 0.19·Radio + 0.001·Newspaper` : the coefficient 0.046 means every $1,000 extra on TV ads adds 46 units to sales. The 0.001 for newspaper is near zero — it barely contributes. This is how you read which features matter.

Assumptions

- **Linearity** — the relationship between X and y is actually linear
- **Independence** — observations don't influence each other
- **Homoscedasticity** — residuals have constant variance across X (no funnel shape in residual plot)
- **Normal residuals** — residuals are approximately normally distributed
- **No multicollinearity** — input features are not highly correlated with each other

> *When assumptions are violated the model still runs, but the coefficient estimates and confidence intervals become unreliable. Always check the residual plot.*

05 Gradient Descent

Linear regression has a closed-form solution (the normal equation) so you could solve it analytically. But for larger datasets and other models, that's computationally expensive. Gradient descent is the iterative alternative — it finds the minimum of the loss function one step at a time.

The analogy that actually makes sense: you're blindfolded on a hilly landscape. You can only feel the slope under your feet. You take one step in the steepest downhill direction, then stop and feel again. Repeat until the ground is flat. You've reached the bottom — the minimum loss.

STEP 1

Initialize

Random values for m and c

STEP 2

Compute loss

Calculate MSE with current params

STEP 3

Get gradient

Slope of loss at this point (∂L/∂θ)

STEP 4

Update params

θ = θ − α · gradient

REPEAT

Until converged

Loss stops dropping

UPDATE RULE

$$\theta = \theta - \alpha \cdot \partial L/\partial\theta$$

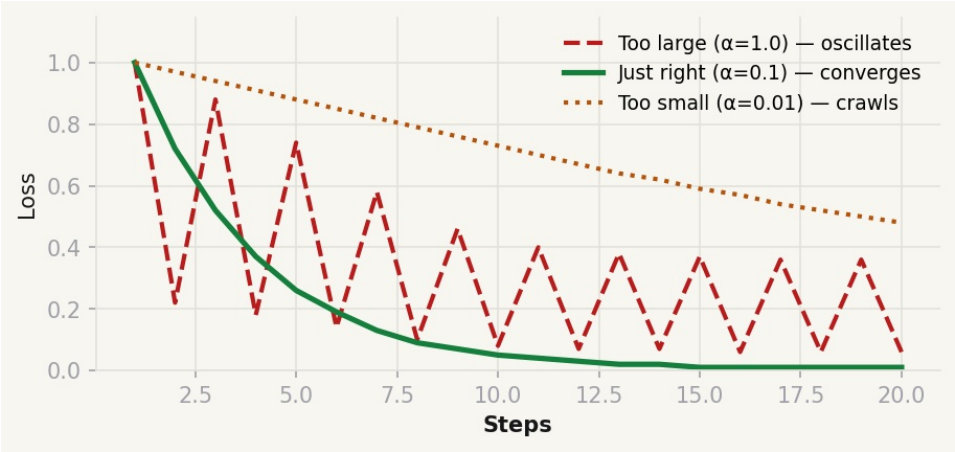
θ — parameter being updated · **α** — learning rate (step size) · **∂L/∂θ** — gradient of the loss

Learning rate

The learning rate α controls how big each step is. It's one of the most important hyperparameters to tune. Too large and you overshoot the minimum — the loss bounces around and never settles. Too small and convergence takes forever. A good learning rate shows the loss dropping steadily and flattening out.

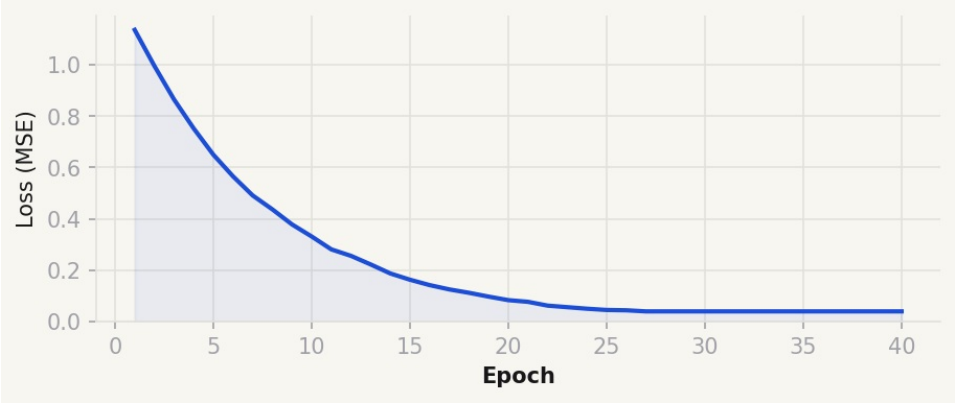
Three learning rates compared

All three start at the same loss. The difference is entirely from the step size.



Loss curve over training epochs

Drops fast at first as the model makes large corrections, then levels off near the minimum



06 Evaluation Metrics

These are the standard ways to measure how good a regression model is. You compute them on the test set — never the training set.

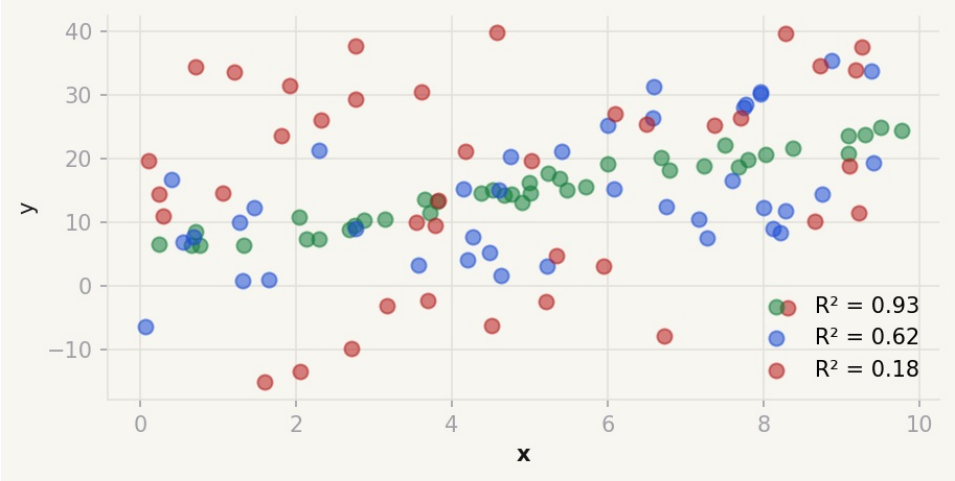
METRIC	FORMULA	WHAT IT TELLS YOU	WANT
MSE	`(1/n) Σ(y − ŷ)²`	Mean squared error. Penalizes large errors heavily due to squaring.	Lower
RMSE	`√MSE`	Root MSE. Same units as the output — more interpretable than MSE.	Lower
MAE	`(1/n) Σ|y − ŷ|`	Mean absolute error. Treats all errors equally. More robust to outliers.	Lower
R^2	`1 − SS_res / SS_tot`	Proportion of variance in the output explained by the model. 0 to 1.	Closer to 1
Adjusted R^2	Penalizes extra features	R^2 corrected for number of features. Use this when comparing models.	Closer to 1

R^2 is the most commonly reported metric. $R^2 = 0.85$ means the model accounts for 85% of the variation in the output. The remaining 15% is noise or features not in the model. R^2 of 1.0 is perfect. R^2 of 0 means the model is no better than just predicting the mean every time.

> **The R^2 problem**
> R^2 always increases when you add more features — even if they're random noise. A model with 50 useless features will report a higher R^2 than one with 5 useful ones. Always use Adjusted R^2 when comparing models with different numbers of features. It only increases if the added feature genuinely improves predictions.

What different R^2 values look like — scatter vs fitted line

Higher R^2: points cluster tightly around the line. Lower R^2: points scatter widely — the line barely captures the pattern.



> *When to use MAE vs RMSE: if outliers are meaningful and should be penalized, use RMSE (it squares errors, amplifying outlier impact). If outliers are noise or measurement errors, use MAE.*

07 Regularization

When a model is too complex — too many features, too high-degree polynomial — it starts fitting the noise in the training data, not just the signal. This is overfitting. Regularization fixes this by adding a penalty for large coefficients directly into the loss function. Larger coefficients = more complexity = higher penalty.

Ridge

L2 REGULARIZATION

$$MSE + \lambda \cdot \Sigma(\beta^2)$$

Penalizes sum of *squared* coefficients. Shrinks all of them toward zero but never to zero. All features stay in the model.

Use when all features contribute something.

Lasso

L1 REGULARIZATION

$$MSE + \lambda \cdot \Sigma|\beta|$$

Penalizes sum of *absolute values*. Can push coefficients to exactly zero — removes features entirely. Performs automatic feature selection.

Use when only some features matter.

Elastic Net

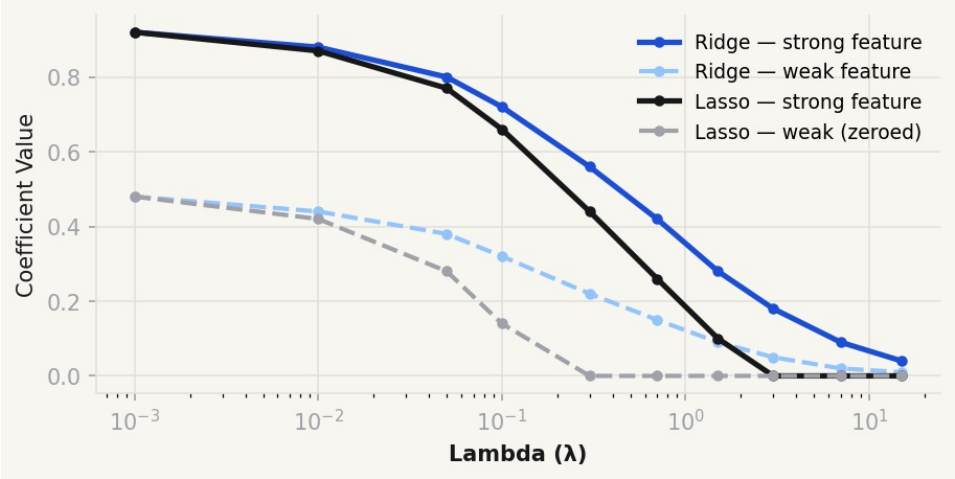
L1 + L2

$$MSE + \lambda_1|\beta| + \lambda_2\beta^2$$

Combines both penalties. Handles correlated features while still performing feature selection. Best of both.

Use for high-dim or correlated features.

Lambda (λ) controls how strong the penalty is. Higher λ → simpler model → less overfitting but potentially higher bias. Lower λ → weaker penalty → closer to plain linear regression. λ is a hyperparameter — tuned with cross-validation.

Coefficient values as regularization strength (λ) increases

Ridge: all coefficients shrink slowly together. Lasso: weak features hit zero first and are removed — stronger features survive longer.